```c
#include <stdio.h>

int firstZero(int arr[], int low, int high, int m) {
    if (low > high)
        return m;               // no zero found

    int mid = low + (high - low) / 2;

    if (arr[mid] == 0) {
        // first zero is at mid if the previous element is 1 (or mid is index 0)
        if (mid == 0 || arr[mid - 1] == 1)
            return mid;
        return firstZero(arr, low, mid - 1, m);
    }
    return firstZero(arr, mid + 1, high, m);
}

int main() {
    int m;
    scanf("%d", &m);

    int arr[m];
    for (int i = 0; i < m; i++)
        scanf("%d", &arr[i]);

    int idx = firstZero(arr, 0, m - 1, m);
    printf("%d\n", m - idx);

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔

✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int candidate = 0, count = 0;
    for (int i = 0; i < n; i++) {
        int x;
        scanf("%d", &x);

        if (count == 0)
            candidate = x;

        if (x == candidate)
            count++;
        else
            count--;
    }

    printf("%d\n", candidate);
    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

```c
#include <stdio.h>
int floorIndex(int arr[], int low, int high, int x) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == x)
        return mid;

    if (arr[mid] > x)
        return floorIndex(arr, low, mid - 1, x);
    int idx = floorIndex(arr, mid + 1, high, x);
    return (idx != -1) ? idx : mid;
}
int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int x;
    scanf("%d", &x);

    int idx = floorIndex(arr, 0, n - 1, x);
    if (idx == -1)
        printf("-1\n");
    else
        printf("%d\n", arr[idx]);

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

```c
#include <stdio.h>

// Divide and conquer: recursive binary search for target in arr[low..high]
int search(int arr[], int low, int high, int target) {
    if (low > high)
        return 0;

    int mid = low + (high - low) / 2;

    if (arr[mid] == target)
        return 1;
    if (arr[mid] > target)
        return search(arr, low, mid - 1, target);
    return search(arr, mid + 1, high, target);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int x;
    scanf("%d", &x);

    for (int i = 0; i < n - 1; i++) {
        int need = x - arr[i];
        if (search(arr, i + 1, n - 1, need)) {
            printf("%d\n%d\n", arr[i], need);
            return 0;
        }
    }

    printf("No\n");
    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

```c
#include <stdio.h>
#include <string.h>

#define MAX 1001

int dp[MAX][MAX];

int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    char s1[MAX], s2[MAX];
    scanf("%1000s", s1);
    scanf("%1000s", s2);

    int m = strlen(s1);
    int n = strlen(s2);

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                dp[i][j] = 0;
            else if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    printf("%d\n", dp[m][n]);
    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.